

Lab3 | Event Driven Architecture with Monitoring and Tracing

This lab is composed of following components, which will ultimately be combined into a complete architecture.

1. Cloud Atlas MongoDB noSQL REST service
2. Setup RabbitMQ Server
3. EDA - Producer-Consumer Queue
4. EDA - Database Event Notification
5. Multi-Module: SQL + NoSQL + Orchestrator
6. Prometheus Monitoring
7. Grafana Visualization
8. Zipkin Tracing

Mandatory Architecture Tasks:

1. Add to the Cloud Atlas noSQL REST service an EDA notification service for a specific POST request
2. Monitoring of the complete application by Prometheus and Grafana

Optional Architecture Tasks:

- 1 Add to the noSQL Atlas - EDA service a SQL service with orchestration of both
- 2 Add **Actuator** Monitoring server for all services
- 3 Add **Eureka** service discovery for all services
- 4 Add a **Gateway** to the application
- 5 Configure a project which uses a connection string to access the MongoDB Atlas Cloud database as a **team-project** to manipulate data
- 6 Containerization with e.g. **Docker** and orchestration with **Kubernetes**
- 7 Replace the in memory H2 by a SQL Cloud service e.g. **Supabase** or **Google Big query**
- 8 Prometheus **Exporter Visualization** of arbitrary metrics
- 9 Add **Zipkin** Tracing for different services
- 10 ...

Source Code for Lab3: <https://github.com/ggaube/swd24-software-architecture>

Task 1 Atlas MongoDB noSQL Cloud REST Service

Along with relational databases, NoSQL databases are gaining popularity. MongoDB is a popular NoSQL database that stores the data as documents in JSON-like format. Spring Boot provides with MongoDB Compass a local and with MongoDB Atlas a cloud based approach to use a noSQL database.

Configuration Tasks for MongoDB Atlas

1. Configure a sign in at: <https://www.mongodb.com/products/platform/atlas-database>
2. Configure Network and Database access
3. Configure Cluster and Connection
4. Test the connection from your client to your cluster

Coding Tasks

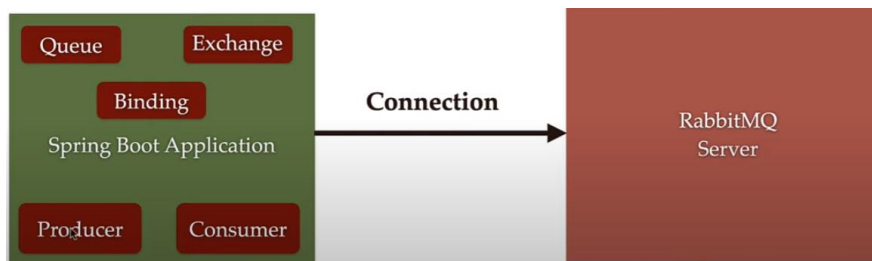
1. Check the configuration in the `application.properties` file and modify your connection string
 - a. Cloud based MongoDB Atlas
 - b. Local Compass MongoDB
2. Check all Java classes (Lombok Plugin is needed)
3. Setup and test the service with an arbitrary connection string
4. Check the Swagger UI, test some requests and check the results in the database
<http://localhost:5000/swagger-ui/index.html>

Task 2: Setup RabbitMQ Server

In a microservices architecture, different services sometimes need to communicate asynchronously. Using event-driven architecture, services can

1. publish **events** when they perform actions or when their state changes,
2. other services can **subscribe** to these events to perform their own actions.

This communication concept by RabbitMQ provides a robust and scalable way to handle **asynchronous communication** and enables to decouple services for various use cases. The RabbitMQ Server provides a performant message queue to enable messaging between a producer and a consumer.



Setup RabbitMQ

1. Install RabbitMQ server: <https://www.rabbitmq.com/docs/download>
2. Install Erlang library for real-time-queuing: <https://www.erlang.org/patches/otp-25.3.2>
3. To enable RabbitMQ management plugin for the UI console, navigate to the directory:
C:\Program Files\RabbitMQ Server\rabbitmq_server-3.13.1\sbin>
rabbitmq-plugins.bat enable rabbitmq_management
4. Check the RabbitMQ service on your computer and allow the access from a local application:
 - a. Check the RabbitMQ dashboard: <http://localhost:15672>
 - b. Login: user / user

Task 3: EDA - Producer-Consumer Queue

This application publishes and subscribes messages to a RabbitMQ AMQP server with a single or a double queue.

Advanced Message Queuing Protocol (AMQP):

- Producers do not send messages directly to **queues**
- Message is sent to an **exchange**, which can go to a single queue or fan out to multiple queues

RabbitTemplate

- Provides functionalities to send and receive messages with RabbitMQ
- Configures a message listener container
- Declare the queue, the exchange, and the binding between them
- Spring Boot automatically creates a connection factory and a *RabbitTemplate*, reducing the amount of code and enables to connect to the RabbitMQ server

RabbitMQConfig

This class sets up the necessary infrastructure to enable communication between different parts of the system using RabbitMQ. Producers can send messages to the exchange with a specific routing key, and consumers can receive messages from the bound queue.

- **Queue:** A storage area for messages waiting to be processed, *notificationQueue* is create
- **Exchange:** A message routing agent takes messages from producers and routes them to queues based on the message's routing key. A *TopicExchange* named *notificationExchange* is created.
- **Binding:** A link between a queue and an exchange that specifies how messages should be routed. Messages sent to the *notificationExchange* with the routing key *notificationQueue* are routed to the *notificationQueue*.

Coding Task

1. Load and start the project `event-driven-prod-con`
2. Check the most important parts of the RabbitMQ communication code
3. Send a message with Postman to the endpoint: <http://localhost:8080/messages> or with Swagger UI <http://localhost:8080/swagger-ui/index.html>
4. Check the output in the Consumer console

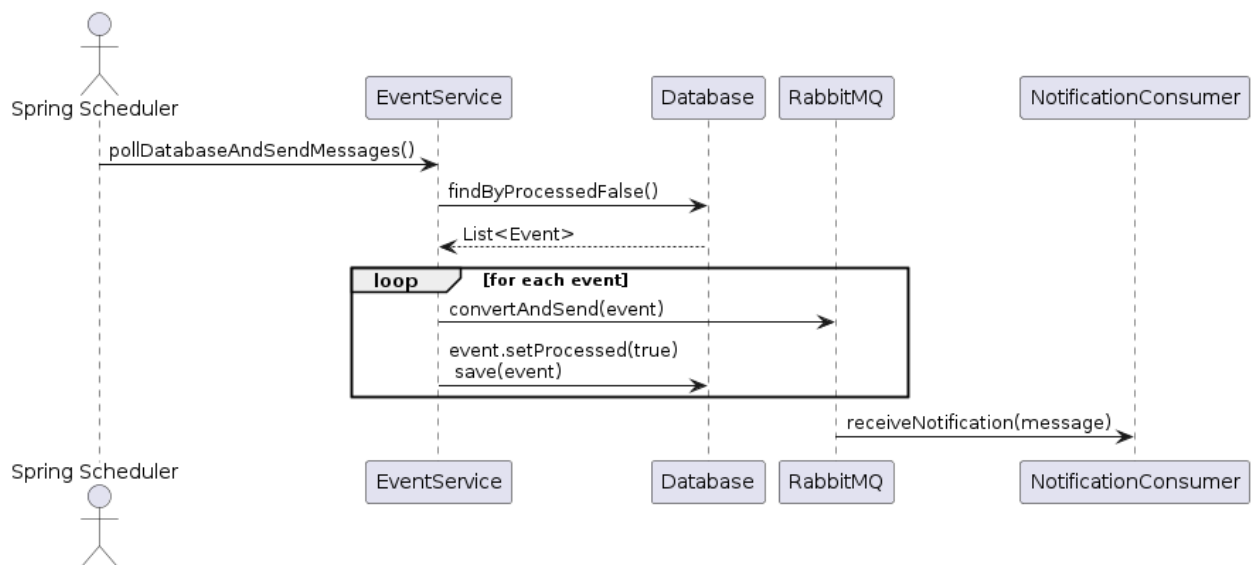
Task 4: EDA - Database Event Notification

This setup ensures that database events are automatically picked up and sent to RabbitMQ without manual intervention. The approach is to use a **database trigger** or **polling** mechanism to detect changes in the database and then publish messages to RabbitMQ based on those change.

Using H2 for development can simplify setup and eliminates the need for an external database.

Project Code: event-driven-database

Sequence Diagram



Project Code: event-driven-database

1. Start the application
2. Access the H2 console at <http://localhost:8080/h2-console>
3. Insert an event into the events table.

```
INSERT INTO events (message, processed)
VALUES ('Hello from the database!', false);
```

4. Check the notification in the console
5. Additionally, check a post to the endpoint: <http://localhost:8080/database/queueDatabase>

Task 5: Multi-Module: SQL + NoSQL + Orchestrator

Project involving inter-service communication and orchestration of these modules:

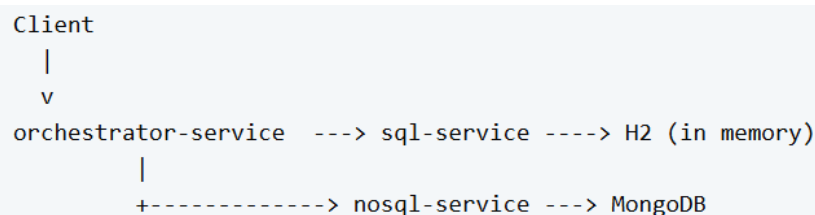
- **sql-service:** H2 in-memory database with relational master data such as *Customer*
- **nosql-service:** MongoDB Compass/Atlas with flexible document data such as *Preferences*
- **orchestrator-service:** as an aggregator that calls both services via *OpenFeign*. It encapsulates the inter-service communication and provides the client with a composite API model.

Workflow

- 1 **HTTP request combines data** from SQL and NoSQL. Spring Boot with REST provides HTTP services, Spring Data JPA covers relational stores, Spring Data MongoDB supports NoSQL MongoDB, and Spring Cloud OpenFeign is a declarative HTTP client for service-to-service communication.
- 2 **OpenFeign allows declarative HTTP clients** via interfaces with Spring MVC annotations; for orchestration between services, this is the most direct approach.
- 3 **Spring Web + Data + Feign design:** prevents a single service from simultaneously orchestrating two completely different persistence models and database connections.

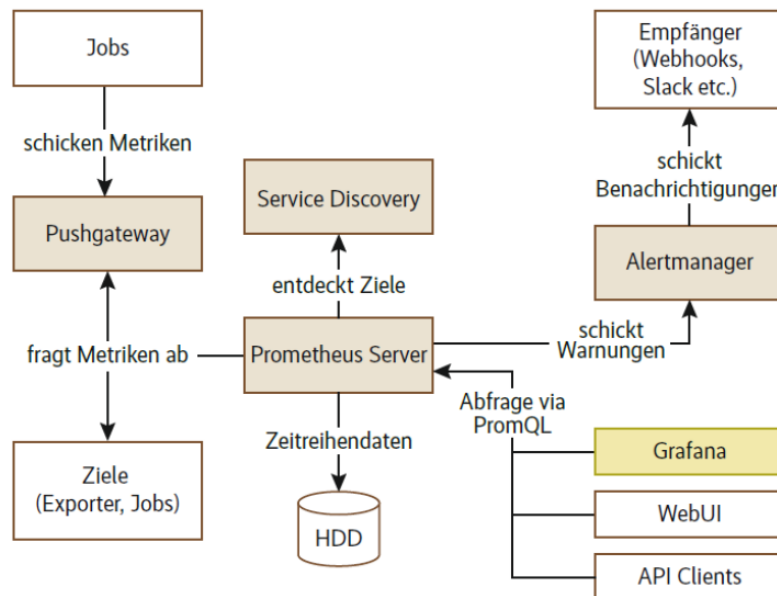
Architecture

API Gateway vs. Orchestrator: this solution is more of an orchestrator / aggregator service than a classic API / Edge Gateway.



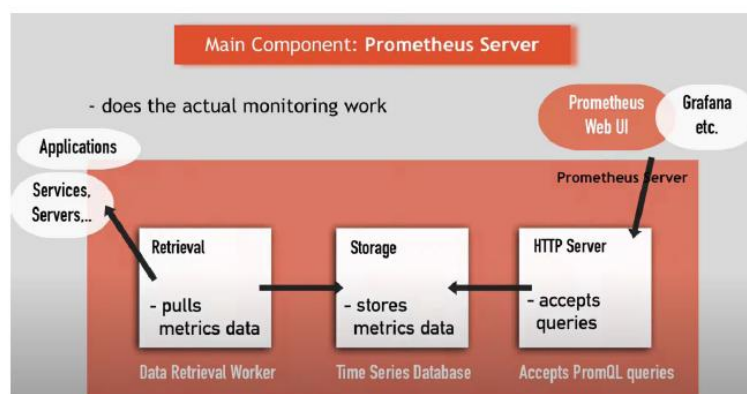
Task 6: Prometheus Monitoring

In this section we will integrate the monitoring system *Prometheus* and the graphic dashboard system *Grafana* to be able to visualize all the metrics generated from the Spring Boot application.



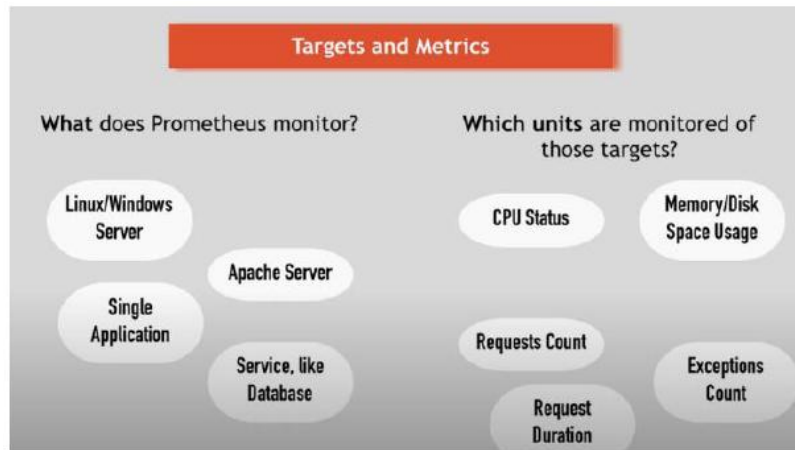
Prometheus is an open-source monitoring system and consists of following components:

- **Data scraper** pulls metrics over HTTP periodically
- **Time-series database** to store all metrics data
- **Dashboards** to visualize, query, and monitor all metrics



Micrometer is a library powering the delivery of application metrics:

- Integrates actuator metrics with external monitoring systems
- Collects and exports metrics in a format that can be scraped by Prometheus server
- Supports several monitoring systems like Netflix, AWS Cloudwatch, Datadog, etc.



Prometheus Server Configuration

Server Installation

- Download server at: <https://prometheus.io/download/>
- Or as docker image
- Various exporter for various metrics are available, e.g. `mysql_exporter` which connects to a MySQL database to export server performance metrics.

Server Configuration

- Edit and adapt `prometheus.yml` in Prometheus file directory
- Basic configurations of global and scrape
- Most important is `job_name` inside `scrape_configs` section
- `metrics_path` is the path to the prometheus endpoint
- Targets section contains the HOST and PORT of the application

Spring-Boot Application Configuration

- Dependencies the POM file
 - `micrometer-registry-prometheus`
 - `spring-boot-starter-actuator`

Metrics Visualization with Prometheus

- All metrics can be found at Prometheus Endpoints:
<http://localhost:8080/actuator/prometheus>
- All metrics can be visualized Prometheus at the Prometheus dashboard:
<http://localhost:9090/>
 - Choose Graph panel
 - Set time window 5 min
- Some example Prometheus metrics to visualize as a graph
 - CPU usage: `system_cpu_usage,`
`http_server_requests_seconds_max`
 - Other Requests: `prometheus_http_requests_total,`
`http_server_requests_seconds_count,`
`jvm_threads_states_threads`

Task 7: Grafana Visualisation

Prometheus provides basic visualization. Grafana imports data from Prometheus and visualizes them with perfect graphs and Dashboards:

- Free Dashboards can be imported as JSON files
- Alert rules based on metrics can be set and notification is done over various channels
- Also other data sources than Prometheus are supported, such as Elasticsearch, InfluxDB, etc

Grafana Installation

- ▪ As Docker image or download as a service:
<https://grafana.com/grafana/download?platform=windows>
- ▪ Navigate to <http://localhost:3000>
 - username *admin*
 - password *admin*

Grafana Configuration

1. Add Data Source

- Import metrics from Prometheus and visualize them on Grafana
- Choose Prometheus as data source
- Add HTTP URL <http://localhost:9090/>
- No additional configuration is needed
- Save & Test and go back to Home Dashboard

2. Add a new Panel

- Data source Prometheus and metric selection. For testing choose `system_cpu_usage` and `system_cpu_count`
- ▪ Change time range
- Test various Visualization graphs - check Heatmap as a very helpful feature for failure diagnostics
- Save the Panel as a Dashboard

3. Import a Dashboard

- Dashboards are available here: <https://grafana.com/grafana/dashboards>
- Filter data source by Prometheus
- Import and test these dashboards:
 - JVM and System: Dashboard ID 4701
 - Prometheus 2.0 Overview: Dashboard ID 3662
 - Prometheus All Metrics: Dashboard ID 19268

Task 8: Gateway Zipkin Tracing

In Zipkin, spans are the smallest units of work within a distributed trace. Each span represents a single operation or step performed while processing a request.

1. The Gateway receives the request
2. The Gateway immediately forwards the request to the Order Service
3. The Order Service instantly starts processing the REST endpoint
4. The business logic (load-order) starts shortly afterward
5. Most of the execution time is spent inside load-order

Span Hierarchy and Request Flow



Four Spans in our Application

Span	Service	Meaning
api-gateway: http get	API Gateway	Incoming HTTP request reaches the gateway
api-gateway: http get	API Gateway	Gateway forwards/routes the request
order-service: http get /orders/{id}	Order Service	REST endpoint is invoked
order-service: load-order	Order Service	Business logic loads the order

Parent and Child Spans

Zipkin organizes spans as a tree structure and each span contains:

- Trace ID (same for all spans in the trace)
- Span ID
- Parent Span ID
- Start time
- End time
- Duration
- Tags (HTTP method, URL, status code, etc.)

